SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Tic Tac Toe Game System

Author: Jun Y. Ercia

Version: 3.0

Date: 2026

Standard: ISO/IEC/IEEE 29148

TABLE OF CONTENTS

1. INTRODUCTION 4
- **1.1 Purpose** 4
- **1.2 Scope** 4
- **1.3 Definitions, Acronyms, and Abbreviations** 4
- **1.4 References** 5

2. OVERALL DESCRIPTION 5
- **2.1 Product Perspective** 5
- **2.2 Product Functions (High-Level Summary)** 5
- **2.3 User Classes and Characteristics** 5
- **2.4 Operating Environment** 6

3. SYSTEM ARCHITECTURE 6
- **3.1 Architectural Overview** 6
- **3.2 Design Principles Applied** 7

4. FUNCTIONAL REQUIREMENTS 8
- **4.1 Game Initialization Requirements** 8
- **4.2 Input Handling Requirements** 9
- **4.3 Game Execution Requirements** 9
- **4.4 Win Detection Requirements** 9
- **4.5 Draw Detection Requirements** 9

5. EXTERNAL INTERFACE REQUIREMENTS 9
- **5.1 User Interface** 10
- **5.2 Hardware Interface** 10
- **5.3 Software Interface** 10

6. NON-FUNCTIONAL REQUIREMENTS 10
- **6.1 Performance** 10
- **6.2 Reliability** 10
- **6.4 Portability** 10

7. DATA REQUIREMENTS 10

8. SYSTEM MODELS 11
- **8.1 UML Class Diagram** 11
- **8.2 Use Case Model** 11

9. DESIGN CONSTRAINTS 12

10. ASSUMPTIONS AND DEPENDENCIES 12

11. REQUIREMENTS TRACEABILITY MATRIX (RTM) 12

12. VERIFICATION AND VALIDATION PLAN 12
13. RISK ANALYSIS 13
14. FUTURE ENHANCEMENTS 13

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the **Tic Tac Toe Game System**, a console-based application developed using Object-Oriented Programming (OOP) principles in Python.

This document establishes:

- System behavior
- Constraints
- Quality attributes
- Validation methods
- Traceability of requirements

This SRS serves as the authoritative reference for development, testing, evaluation, and academic assessment.

1.2 Scope

The Tic Tac Toe Game System is a standalone interactive console application that enables two human players to play the classic 3×3 grid game.

The system shall:

- Initialize a 3×3 board
- Register two players
- Enforce turn-taking
- Validate input
- Detect win and draw conditions
- Terminate properly

The system shall not:

- Support artificial intelligence opponents
- Store persistent data
- Support multiplayer over network
- Provide graphical user interface

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
OOP	Object-Oriented Programming
Slot	A cell in the 3×3 grid
Chip	Symbol assigned to player
Board	3×3 grid container
Controller	Game flow manager

1.4 References

- ISO/IEC/IEEE 29148:2018 Systems and Software Engineering Requirements
- IEEE 830-1998 Software Requirements Specification
- Python 3.x Official Documentation

2. OVERALL DESCRIPTION

2.1 Product Perspective

The system is an independent, console-based application.

Architectural characteristics:

- Layered structure
- Controller pattern
- Encapsulation
- Modular design
- Single Responsibility Principle

2.2 Product Functions (High-Level Summary)

The system shall:

1. Register players
2. Initialize board
3. Manage turns
4. Validate input
5. Detect winner
6. Detect draw
7. Display results
8. Terminate gracefully

2.3 User Classes and Characteristics

The Tic Tac Toe Game System is intended for general users who want to play a two-player console-based game.

2.3.1 General Users

General users are individuals who interact with the system for gameplay purposes.

Characteristics:

- Have a basic understanding of Tic Tac Toe rules
- Can operate a computer and use a keyboard
- Can enter numeric input in a command-line environment
- Do not require programming knowledge

The system provides clear prompts, simple instructions, and easy-to-understand feedback to support this user group.

2.4 Operating Environment

- Python 3.x
- Windows / macOS / Linux
- Command-line interface

3. SYSTEM ARCHITECTURE

3.1 Architectural Overview

The Tic Tac Toe Game System follows a modular, layered architecture that separates responsibilities into clearly defined components. This structure promotes maintainability, scalability, clarity, and low coupling between modules.

The system is organized into four primary logical layers:

1. **Presentation Layer (User Interaction Layer)**
   This layer represents the console-based interaction between the user and the system. The user provides numeric input corresponding to board positions and receives feedback regarding game status, errors, and results.
2. **Controller Layer (TicTacToeGame Class)**
   The Controller Layer manages the overall execution of the game. It is responsible for:
   - Initializing system components
   - Managing player turns
   - Validating move flow
   - Coordinating interactions between layers
   - Determining termination conditions
3. **Game Logic Layer (Board Class)**
   The Game Logic Layer enforces the core rules of the game. It is responsible for:
   - Maintaining the 3×3 grid
   - Checking slot availability
   - Evaluating win conditions
   - Detecting draw conditions
4. **Data Model Layer (Slot and Chip Classes)**
   The Data Model Layer represents the internal data structures of the system.
   - The `Slot` class stores the state of each board cell and checks availability.
   - The `Chip` class stores the player's assigned symbol (X or O).

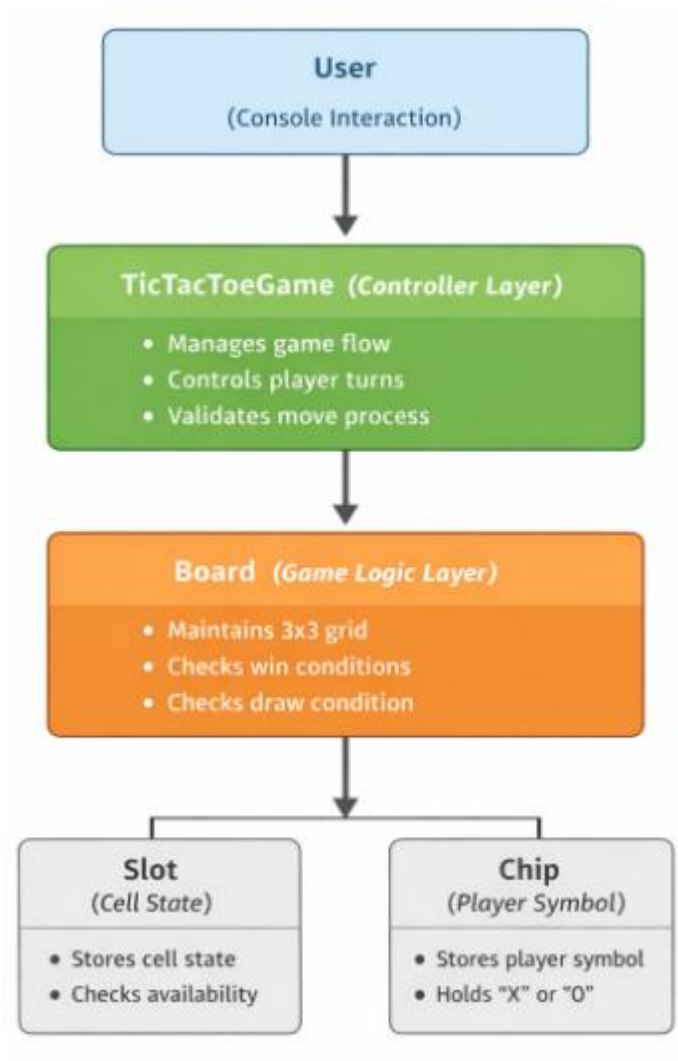
The overall interaction flow follows a top-down structure:

User → Controller → Game Logic → Data Model

Each layer communicates only with adjacent layers, ensuring modular separation and controlled dependency.

The architectural design of the system is illustrated in Figure 1.

Figure 1. Layered architecture of the Tic Tac Toe Game System.



3.2 Design Principles Applied

The architectural design of the Tic Tac Toe Game System is based on established object-oriented design principles. These principles guide the organization of system components and support maintainability, clarity, and controlled system evolution.

Encapsulation

The system organizes data and behavior within defined classes. Each class manages its internal state and exposes functionality through clearly defined public methods. Internal data structures, such as the board grid and player symbols, are maintained within their respective classes to ensure controlled access and integrity of state.

Information Hiding

Implementation details are concealed from external components. Internal attributes are not directly accessed outside their owning class. Interaction between components occurs only through defined interfaces. This approach reduces system complexity and minimizes the impact of internal changes on other components.

Low Coupling

The system structure minimizes dependencies between classes. Each layer interacts only with adjacent layers and does not directly manipulate the internal structure of other components. This design reduces interdependency, simplifies maintenance, and supports future extension.

High Cohesion

Each class has a clearly defined and focused responsibility. The Controller manages game flow, the Board enforces game rules, the Player manages input, and the Slot and Chip classes represent data elements. Concentrating related responsibilities within single classes improves clarity and system organization.

GRASP – Information Expert Principle

Responsibility assignment follows the GRASP Information Expert principle. Functions are assigned to the class that possesses the necessary information to perform them. For example, the Board class is responsible for evaluating win and draw conditions because it maintains the grid data required for those computations. This ensures logical responsibility distribution and reduces unnecessary communication between classes.

4. FUNCTIONAL REQUIREMENTS

4.1 Game Initialization Requirements

FR-1 The system shall initialize a 3×3 board containing nine empty slots upon game start.

FR-2 The system shall prompt for Player 1 name before the game begins.

FR-3 The system shall prompt for Player 2 name before the game begins.

FR-4 The system shall assign symbol “X” to Player 1.

FR-5 The system shall assign symbol “O” to Player 2.

FR-6 The system shall designate Player 1 as the starting player.

4.2 Input Handling Requirements

FR-7 The system shall prompt the active player to enter a numeric slot between 1 and 9.

FR-8 The system shall validate that input is an integer.

FR-9 The system shall reject non-numeric input and display an error message.

FR-10 The system shall reject values outside the range 1–9.

FR-11 The system shall verify that the selected slot is available.

FR-12 The system shall reject selection of occupied slots.

4.3 Game Execution Requirements

FR-13 The system shall place the player's chip in the validated slot.

FR-14 The system shall display the updated board after each valid move.

FR-15 The system shall alternate the active player after each valid move.

FR-16 The system shall continue gameplay until a termination condition occurs.

4.4 Win Detection Requirements

FR-17 The system shall detect horizontal win conditions.

FR-18 The system shall detect vertical win conditions.

FR-19 The system shall detect main diagonal win conditions.

FR-20 The system shall detect secondary diagonal win conditions.

FR-21 The system shall display the winning player's name upon win detection.

FR-22 The system shall terminate the game immediately after detecting a win.

4.5 Draw Detection Requirements

FR-23 The system shall detect when all nine slots are occupied without a win.

FR-24 The system shall declare the game a draw when no win exists.

FR-25 The system shall terminate the game after declaring a draw.

5. EXTERNAL INTERFACE REQUIREMENTS

5.1 User Interface

The system shall:

- Provide text-based prompts
- Provide clear error feedback
- Display board in structured 3×3 format

5.2 Hardware Interface

The system shall operate on any computer supporting Python 3.x.

5.3 Software Interface

The system shall require only the Python Standard Library.

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Performance

NFR-1 The system shall process user input within one second.
NFR-2 The system shall evaluate win conditions in constant time for a 3×3 grid.

6.2 Reliability

NFR-3 The system shall prevent invalid memory or index access.
NFR-4 The system shall prevent overwriting of existing chips.

6.3 Maintainability

NFR-5 The system shall follow modular OOP design.
NFR-6 The system shall separate responsibilities across classes.

6.4 Portability

NFR-7 The system shall execute on Windows, macOS, and Linux platforms.

7. DATA REQUIREMENTS

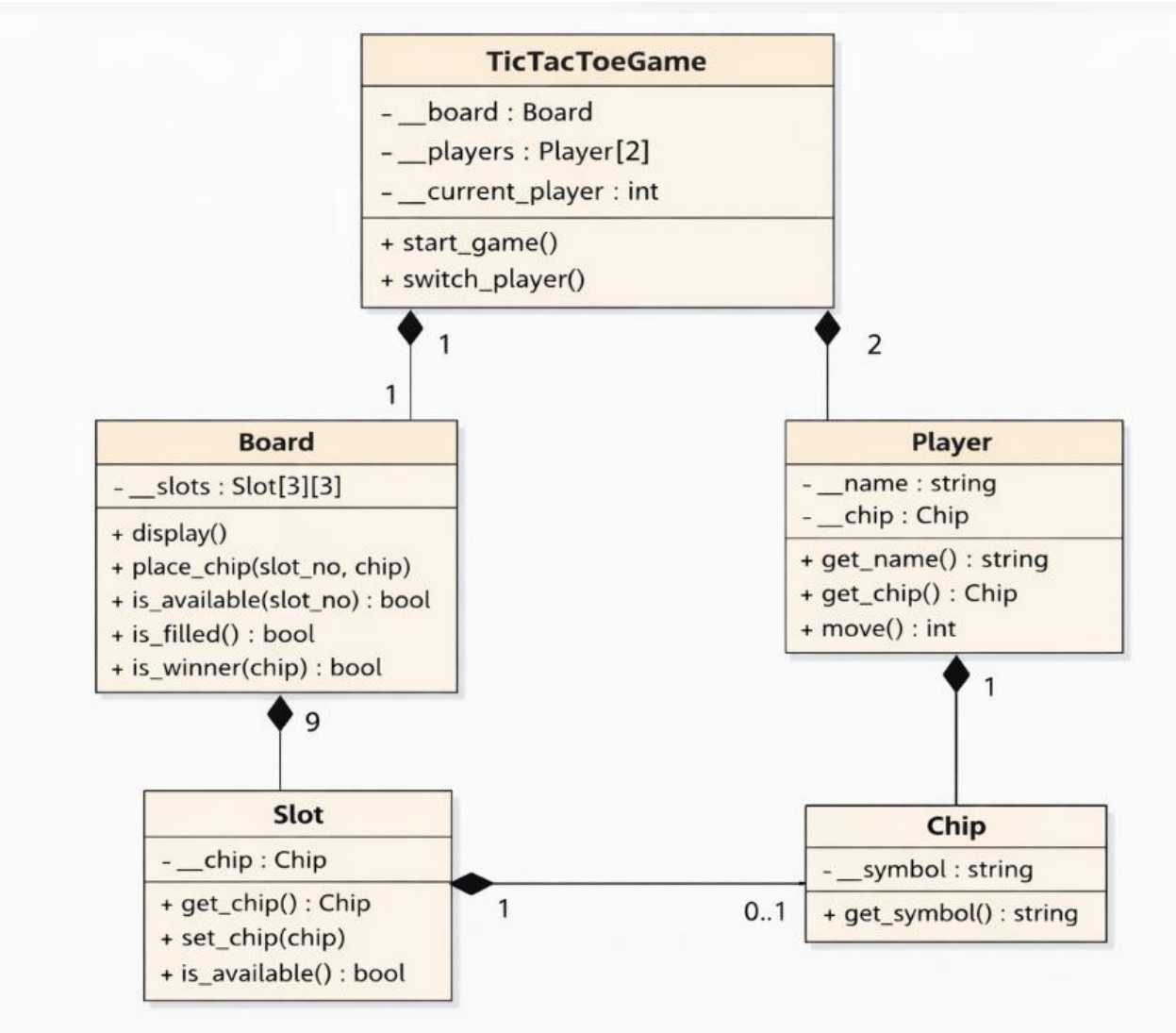
The system shall manage:

- Player name (String)
- Player symbol (Character)
- Slot state (Chip or None)
- Current player index (Integer)
- Board grid (2D list structure)

No persistent data storage required.

8. SYSTEM MODELS

8.1 UML Class Diagram



8.2 Use Case Model

Use Case: Play Tic Tac Toe

Primary Actor: Player

Precondition:

- Game initialized

Main Flow:

1. Player enters move
2. System validates input
3. System updates board

4. System checks win/draw
5. System switches player

Postcondition:

- Game ends or continues

9. DESIGN CONSTRAINTS

- Console-based implementation
- 3×3 fixed grid
- No external libraries
- No AI logic

10. ASSUMPTIONS AND DEPENDENCIES

- Users understand game rules
- Python interpreter available
- Terminal supports standard output

11. REQUIREMENTS TRACEABILITY MATRIX (RTM)

Req ID	Design Component	Test Case
FR-1	Board.**init**	TC-01
FR-11	Board.is_available	TC-02
FR-17	Board.is_winner	TC-03
FR-23	Board.is_filled	TC-04

12. VERIFICATION AND VALIDATION PLAN

Verification Methods:

- Unit Testing
- Manual Testing
- Boundary Testing
- Input Validation Testing

Validation ensures:

- Accurate win detection
- Accurate draw detection
- Stable gameplay execution

13. RISK ANALYSIS

Risk	Impact	Mitigation
Invalid input	Crash	Input validation
Logic error	Wrong winner	Unit testing
Index error	Program failure	Boundary checks

14. FUTURE ENHANCEMENTS

- AI opponent
- GUI interface
- Web-based version
- Score tracking
- Multiplayer network mode